

ERSim

Emergency Room Simulation in R

Akhil Ambalapurath Ajith
David Abraham
Mevin Biju

PROJECT OVERVIEW

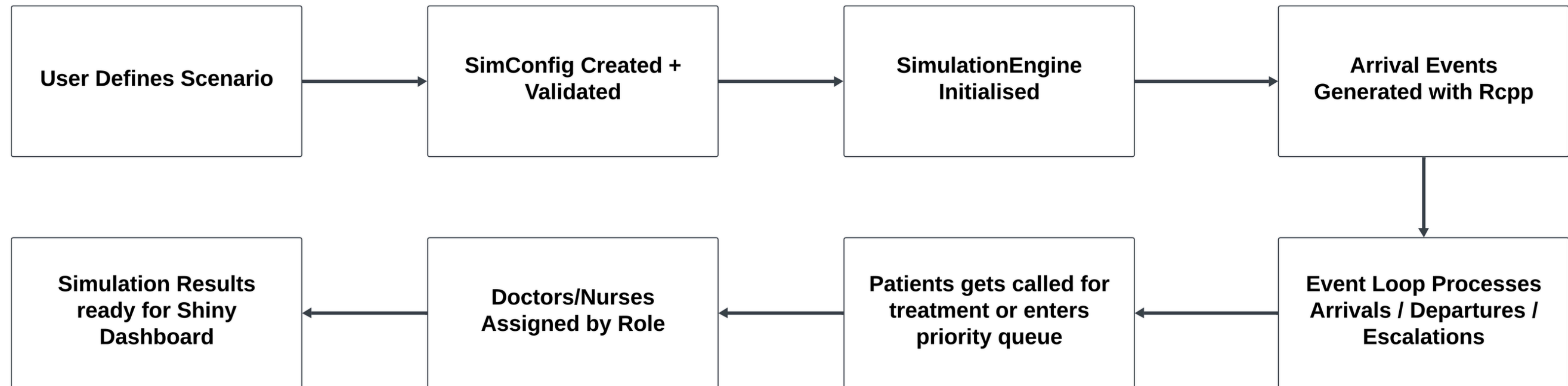
ERSim simulates how patients move through an Emergency Room system.

It models:

- Patient Arrivals
- Urgency Levels
- Staff Allocation
- Priority Queues
- Patient Condition Escalation
- Dashboard Visualization



MAIN SIMULATION FLOW



USER DEFINES SCENARIO

The simulation starts with user-defined assumptions, such as:

- arrival rate
- number of doctors
- number of nurses
- simulation duration
- urgency probabilities
- random seed

```
new_sim_config <- function(arrival_rate = 0.5,  
                             n_doctors   = 3L,  
                             n_nurses   = 5L,  
                             sim_duration = 480,  
                             urgency_probs = c(0.2, 0.3, 0.5),  
                             service_params = list(  
                               "1" = list(mean = 30, sd = 10), # Critical  
                               "2" = list(mean = 20, sd = 7),  # Urgent  
                               "3" = list(mean = 15, sd = 5)   # Standard  
                             ),  
                             seed = NULL) {
```

CREATING AND VALIDATING THE CONFIGURATION

The configuration is checked before the simulation starts. Invalid inputs stop the program early with clear error messages, while valid inputs pass quietly into the simulation engine.

```
validate_sim_config <- function(config) {  
  if (!inherits(config, "SimConfig")) {  
    stop("`config` must be a SimConfig object created by `new_sim_config()`.", call. = FALSE)  
  }  
  assert_positive_number(config$arrival_rate, "arrival_rate")  
  assert_positive_integer(config$n_doctors, "n_doctors")  
  assert_positive_integer(config$n_nurses, "n_nurses")  
  assert_positive_number(config$sim_duration, "sim_duration")  
  assert_probability_vector(config$urgency_probs, "urgency_probs")  
  assert_service_params(config$service_params)  
  if (!is.null(config$seed)) {  
    if (!is.numeric(config$seed) || length(config$seed) != 1L ||  
        is.na(config$seed)) {  
      stop("`seed` must be a single integer or NULL.", call. = FALSE)  
    }  
  }  
  invisible(config)  
}
```

```
assert_probability_vector <- function(probs, name = deparse(substitute(probs)), tol = 1e-6) {  
  if (!is.numeric(probs) || length(probs) != 3L) {  
    stop(sprintf("`%s` must be a numeric vector of length 3.", name), call. = FALSE)  
  }  
  if (any(is.na(probs)) || any(probs < 0)) {  
    stop(sprintf("`%s` must contain non-negative values.", name), call. = FALSE)  
  }  
  if (abs(sum(probs) - 1) > tol) {  
    stop(sprintf("`%s` must sum to 1 (got %.6f).", name, sum(probs)), call. = FALSE)  
  }  
  invisible(probs)  
}
```

```
assert_positive_number <- function(x, name = deparse(substitute(x))) {  
  if (!is.numeric(x) || length(x) != 1L || is.na(x) || x <= 0) {  
    stop(sprintf("`%s` must be a single positive number, got: %s",  
                name, paste(x, collapse = ", ")), call. = FALSE)  
  }  
  invisible(x)  
}
```




INITIALISING THE SIMULATION ENGINE

The simulation engine class is the main orchestrator using R6

Stores the following details:

- priority queue
- doctors
- nurses
- event list
- patient store
- current simulation time

```
SimulationEngine <- R6::R6Class(  
  classname = "SimulationEngine",  
  cloneable = FALSE,  
  
  private = list(  
    config      = NULL,  
    queue       = NULL,  
    doctors     = NULL,  
    nurses      = NULL,  
    all_resources = NULL,  
    event_list  = NULL,  
    patient_store = NULL,  
    patient_counter = 0L,  
    current_time = 0,  
    results     = NULL,  
  )  
)
```

```
initialize = function(config) {  
  validate_sim_config(config)  
  private$config      <- config  
  private$queue       <- PriorityQueue$new()  
  private$doctors     <- make_resources(config$n_doctors, "doctor")  
  private$nurses      <- make_resources(config$n_nurses, "nurse")  
  private$all_resources <- c(private$doctors, private$nurses)  
  private$patient_store <- list()  
  private$event_list  <- data.frame(  
    type      = character(0),  
    time      = numeric(0),  
    patient_id = character(0),  
    urgency_level = integer(0),  
    stringsAsFactors = FALSE  
  )  
  private$patient_counter <- 0L  
  private$current_time    <- 0  
}
```

GENERATING PATIENT ARRIVALS WITH RCPP

Patient arrivals are generated using:

- Exponential inter-arrival times
- Urgency probabilities
- Simulation duration limit

```
DataFrame generate_arrivals_cpp(double arrival_rate,  
                                double sim_duration,  
                                NumericVector urgency_probs) {  
  
    if (arrival_rate <= 0.0)  
        stop("arrival_rate must be positive.");  
    if (sim_duration <= 0.0)  
        stop("sim_duration must be positive.");  
    if (urgency_probs.size() != 3)  
        stop("urgency_probs must have exactly 3 elements.");  
  
    double cp1 = urgency_probs[0];  
    double cp2 = urgency_probs[0] + urgency_probs[1];  
  
    std::vector<double> times;  
    std::vector<int> urgencies;  
    double current_time = 0.0;  
    int expected_n = (int)(arrival_rate * sim_duration * 1.5) + 10;  
    times.reserve(expected_n);  
    urgencies.reserve(expected_n);  
  
    while (true) {  
        double u_time = R::rexp(1.0 / arrival_rate);  
        current_time += u_time;  
  
        if (current_time >= sim_duration) break;  
  
        double u_urgency = R::runif(0.0, 1.0);  
        int urgency;  
        if (u_urgency < cp1) urgency = 1;  
        else if (u_urgency < cp2) urgency = 2;  
        else urgency = 3;  
  
        times.push_back(current_time);  
        urgencies.push_back(urgency);  
    }  
}
```

```
int n = (int)times.size();  
  
CharacterVector type_col(n, "ARRIVAL");  
NumericVector time_col(n);  
CharacterVector pid_col(n, NA_STRING);  
IntegerVector urgency_col(n);  
  
for (int i = 0; i < n; ++i) {  
    time_col[i] = times[i];  
    urgency_col[i] = urgencies[i];  
}  
  
DataFrame result = DataFrame::create(  
    Named("type") = type_col,  
    Named("time") = time_col,  
    Named("patient_id") = pid_col,  
    Named("urgency_level") = urgency_col  
);  
  
return result;  
}
```

```
arrivals_df <- generate_arrivals_cpp(  
    arrival_rate = cfg$arrival_rate,  
    sim_duration = cfg$sim_duration,  
    urgency_probs = cfg$urgency_probs  
)
```

EVENT LOOP PROCESSING

The simulation processes events in chronological order.

Main event types:

- ARRIVAL
- DEPARTURE
- ESCALATION



```
while (nrow(private$event_list) > 0L) {  
  
  idx  <- which.min(private$event_list$time)  
  event <- private$event_list[idx, ]  
  private$event_list <- private$event_list[-idx, ]  
  
  private$current_time <- event$time  
  
  if (event$type == "ARRIVAL") {  
    private$process_arrival(event)  
  } else if (event$type == "DEPARTURE") {  
    private$process_departure(event)  
  } else if (event$type == "ESCALATION") {  
    private$process_escalation(event)  
  }  
}
```


ROLE-BASED RESOURCE ASSIGNMENT

Patients are assigned to appropriate medical staff based on casualty urgency:

- critical and urgent patients go to doctors
- standard patients go to nurses
- escalated patients may move to doctor care

Doctors and nurses are not treated as identical staff to make ERSim's simulation more realistic.

```
process_arrival = function(event) {
  private$patient_counter <- private$patient_counter + 1L
  pid <- make_patient_id(private$patient_counter)

  urgency <- as.integer(event$urgency_level)
  svc_time <- draw_service_time(urgency, private$config$service_params)

  patient <- Patient$new(
    id          = pid,
    arrival_time = event$time,
    urgency_level = urgency,
    service_time = svc_time
  )
  private$patient_store[[pid]] <- patient

  if (urgency %in% c(1L, 2L)) {
    resource <- private$find_available_doctor()
  } else {
    resource <- private$find_available_nurse()
  }
  if (!is.null(resource)) {
    private$serve_patient(patient, resource, event$time)
  } else {
    private$queue$enqueue(patient)
    if (urgency == 3L) {
      esc_time <- event$time + 20.0
      if (esc_time < private$config$sim_duration) {
        private$schedule_event("ESCALATION", esc_time, pid, urgency)
      }
    }
  }
},
```

PATIENTS ENTER THE PRIORITY QUEUE

If no suitable doctor or nurse is available, the patient enters a queue.

The queue orders patients by:

- urgency level
- waiting time

Made into its own R6 class, so the simulation engine class does not need to manage every detail of patient ordering directly. Similarly 'Patient' and 'Resource' are other classes in our project

```
PriorityQueue <- R6::R6Class(  
  classname = "PriorityQueue",  
  cloneable = FALSE,  
  
  private = list(  
    .queue = NULL,  
    tier_entry_time = function(patient, eff_tier, base, rates) {  
      original_tier <- patient$urgency_level  
      if (eff_tier >= original_tier || rates[[as.character(original_tier)]] == 0) {  
        return(patient$arrival_time)  
      }  
  
      patient$arrival_time +  
        (base[[as.character(original_tier)]] - base[[as.character(eff_tier)]] /  
          rates[[as.character(original_tier)]]  
    )  
  ),  
)
```

```
enqueue = function(patient) {  
  if (!inherits(patient, "Patient")) {  
    stop("`patient` must be a Patient R6 object.", call. = FALSE)  
  }  
  private$.queue <- c(private$.queue, list(patient))  
  ord <- order(  
    vapply(private$.queue, function(p) p$urgency_level, integer(1L)),  
    vapply(private$.queue, function(p) p$arrival_time, double(1L))  
  )  
  private$.queue <- private$.queue[ord]  
  invisible(self)  
},
```

DYNAMIC URGENCY AND PATIENT ESCALATION

- Starting priority values by urgency type: Critical - 10, Urgent - 20, Standard -40
- Calculating an effective priority for every queued patient taking into consideration patient wait time to find next patient.
- Leveraging vectorization based functions like `vapply` for performance gain.

```
dequeue_for_doctor = function(current_time = 0) {  
  if (self$is_empty()) return(NULL)  
  
  BASE <- c("1" = 10.0, "2" = 20.0, "3" = 40.0)  
  RATES <- c("1" = 0.0, "2" = 1.0, "3" = 1.0)  
  
  eff <- vapply(private$.queue, function(p) {  
    key <- as.character(p$urgency_level)  
    wait <- max(0.0, current_time - p$arrival_time)  
    max(0.0, BASE[[key]] - RATES[[key]] * wait)  
  }, double(1L))  
  
  # Eligible: Critical + Urgent + Standard that has decayed to eff <= 20  
  eligible <- which(eff <= 20.0)  
  if (length(eligible) == 0L) return(NULL)  
  
  eff_sub <- eff[eligible]  
  eff_tier_sub <- as.integer(ifelse(eff_sub <= 10.0, 1L, 2L))  
  urgency_sub <- vapply(private$.queue[eligible],  
    function(p) p$urgency_level, integer(1L))  
  arrivals_sub <- vapply(private$.queue[eligible],  
    function(p) p$arrival_time, double(1L))  
  tier_entry_sub <- mapply(  
    function(p, tier) private$tier_entry_time(p, tier, BASE, RATES),  
    private$.queue[eligible],  
    eff_tier_sub  
  )  
  idx <- eligible[order(eff_tier_sub, tier_entry_sub, urgency_sub, arrivals_sub)[1L]]  
  
  patient <- private$.queue[[idx]]  
  private$.queue <- private$.queue[-idx]  
  patient  
},
```

SIMULATION RESULTS

```
patient_log      <- private$build_patient_log()
kpis             <- compute_kpis(patient_log)
queue_over_time  <- compute_queue_over_time(patient_log)
queue_ts_analysis <- compute_queue_ts(queue_over_time)
resource_log     <- private$build_resource_log()
elapsed_sec      <- proc.time()[\"elapsed\"] - start_time

private$results <- new_sim_results(
  patient_log      = patient_log,
  kpis             = kpis,
  queue_over_time  = queue_over_time,
  queue_ts_analysis <- queue_ts_analysis,
  resource_log     = resource_log,
  config          = cfg,
  elapsed_sec      = unname(elapsed_sec)
)
```

151

Patients Served



81

Patients Not Served



58.3 min

Mean Wait Time per Served Patient

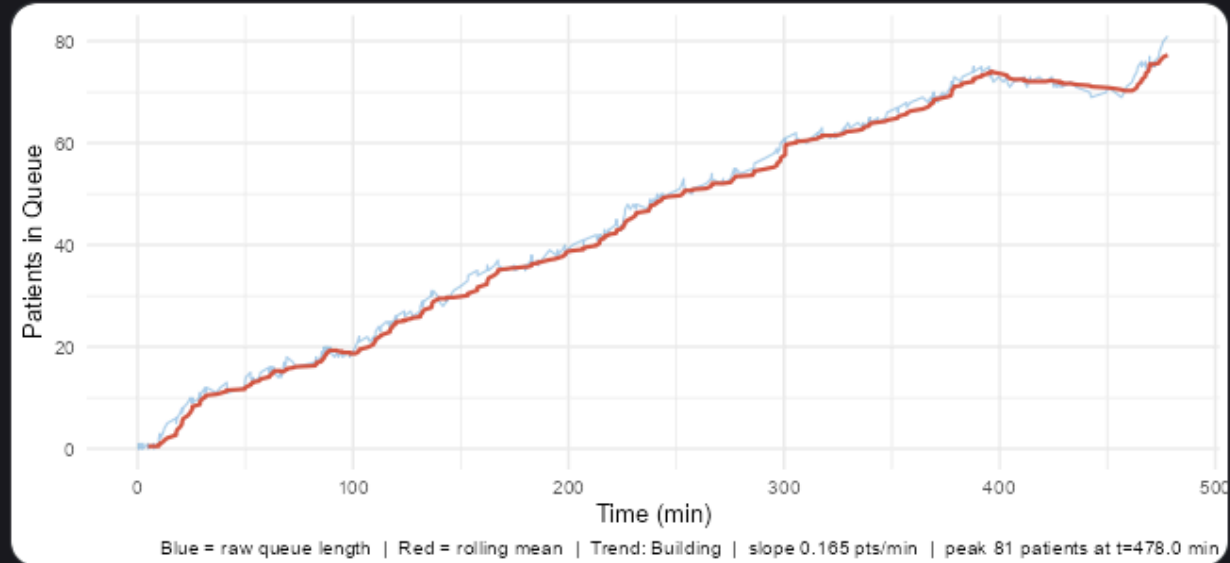


18.9/hr

Throughput



Queue Length Over Time (with Rolling Mean)



Patient Log

Show 10 entries

Search:

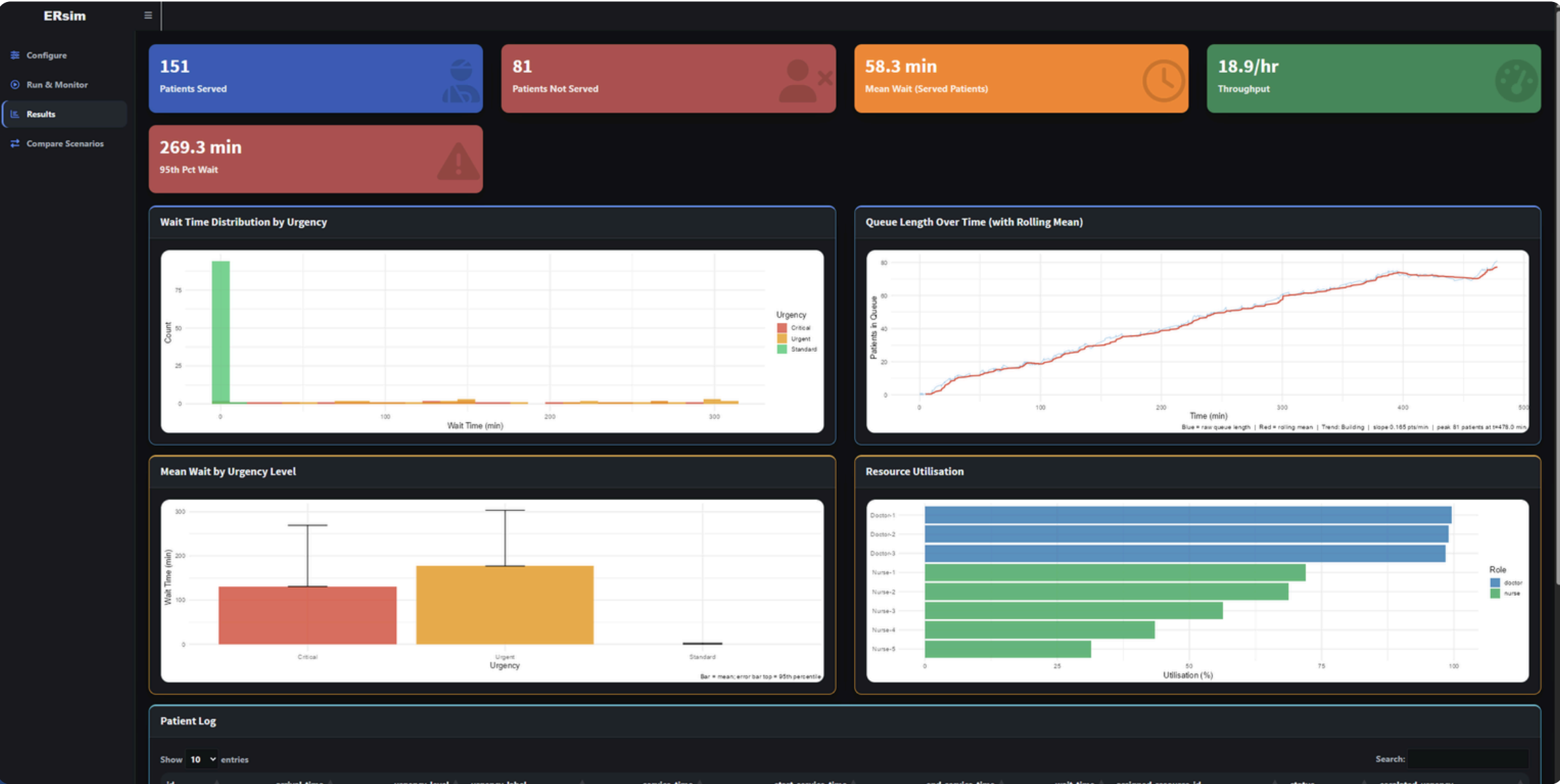
id	arrival_time	urgency_level	urgency_label	service_time	start_service_time	end_service_time	wait_time	assigned_resource_id	status	escalated_urgency
P00001	0.40	3	Standard	15.01	0.40	15.40	0.00	Nurse-1	Discharged	
P00002	0.96	3	Standard	9.53	0.96	10.49	0.00	Nurse-2	Discharged	
P00003	1.91	1	Critical	39.96	1.91	41.87	0.00	Doctor-1	Discharged	
P00004	2.54	3	Standard	12.16	2.54	14.70	0.00	Nurse-3	Discharged	
P00005	4.92	2	Urgent	20.13	4.92	25.05	0.00	Doctor-2	Discharged	
P00006	7.61	1	Critical	28.94	7.61	36.55	0.00	Doctor-3	Discharged	
P00007	9.77	1	Critical	28.46	25.05	53.51	15.28	Doctor-2	Discharged	
P00008	9.89	2	Urgent	34.92	53.51	88.43	43.62	Doctor-2	Discharged	Urgent → Critical
P00009	10.51	3	Standard	10.89	10.51	21.39	0.00	Nurse-2	Discharged	
P00010	11.75	2	Urgent	27.86	63.58	91.44	51.83	Doctor-1	Discharged	Urgent → Critical

Showing 1 to 10 of 232 entries

Previous 1 2 3 4 5 ... 24 Next

SHINY DASHBOARD

Users can change the ER settings, run the model, and instantly see the results as KPIs, queue trends, wait-time plots, and staff utilisation charts.



```
output$plot_queue_time <- renderPlot({
  res <- current_results()
  req(res)
  ts <- res$queue_ts_analysis
  df <- ts$rolling_mean
  lbl <- sprintf("Trend: %s | slope %.3f pts/min | peak %d patients at t=%.1f min",
    ts$trend_label, ts$trend_slope, ts$peak_length, ts$peak_time)
  ggplot(df, aes(x = time)) +
    geom_line(aes(y = queue_length), colour = "#3498db",
      linewidth = 0.5, alpha = 0.45) +
    geom_line(aes(y = rolling_mean), colour = "#e74c3c",
      linewidth = 1.0, na.rm = TRUE) +
    labs(x = "Time (min)", y = "Patients in Queue",
      caption = paste0("Blue = raw queue length | Red = rolling mean | ", lbl)) +
    theme_minimal(base_size = 13)
})
```

```
observeEvent(input$btn_apply_config, {
  total <- input$prob_critical + input$prob_urgent + input$prob_standard
  if (abs(total - 100) > 0.5) {
    showNotification("Urgency probabilities must sum to 100%", type = "error")
    return()
  }

  seed_val <- if (is.na(input$seed) || input$seed == "") NULL else as.integer(input$seed)

  cfg <- tryCatch(
    new_sim_config(
      arrival_rate = input$arrival_rate,
      n_doctors = input$n_doctors,
      n_nurses = input$n_nurses,
      sim_duration = input$sim_duration,
      urgency_probs = c(input$prob_critical,
        input$prob_urgent,
        input$prob_standard) / 100,
      service_params = list(
        "1" = list(mean = input$svc_crit_mean, sd = input$svc_crit_sd),
        "2" = list(mean = input$svc_urg_mean, sd = input$svc_urg_sd),
        "3" = list(mean = input$svc_std_mean, sd = input$svc_std_sd)
      ),
      seed = seed_val
    ),
    error = function(e) {
      showNotification(paste("Config error:", e$message), type = "error")
      NULL
    }
  )

  if (!is.null(cfg)) {
    current_config(cfg)
    showNotification("Configuration applied.", type = "message")
  }
})
```

PACKAGING ERSim

Steps to Package ERSim:

- Organized Modular Code:
 - R files (R) for simulation logic.
 - C++ files (src) for high-performance components
 - Shiny app (shiny) for visualization.
- Documentation with Roxygen2
- Final Packaging:
Used devtools for **check()**, **build()**, and **install()**.

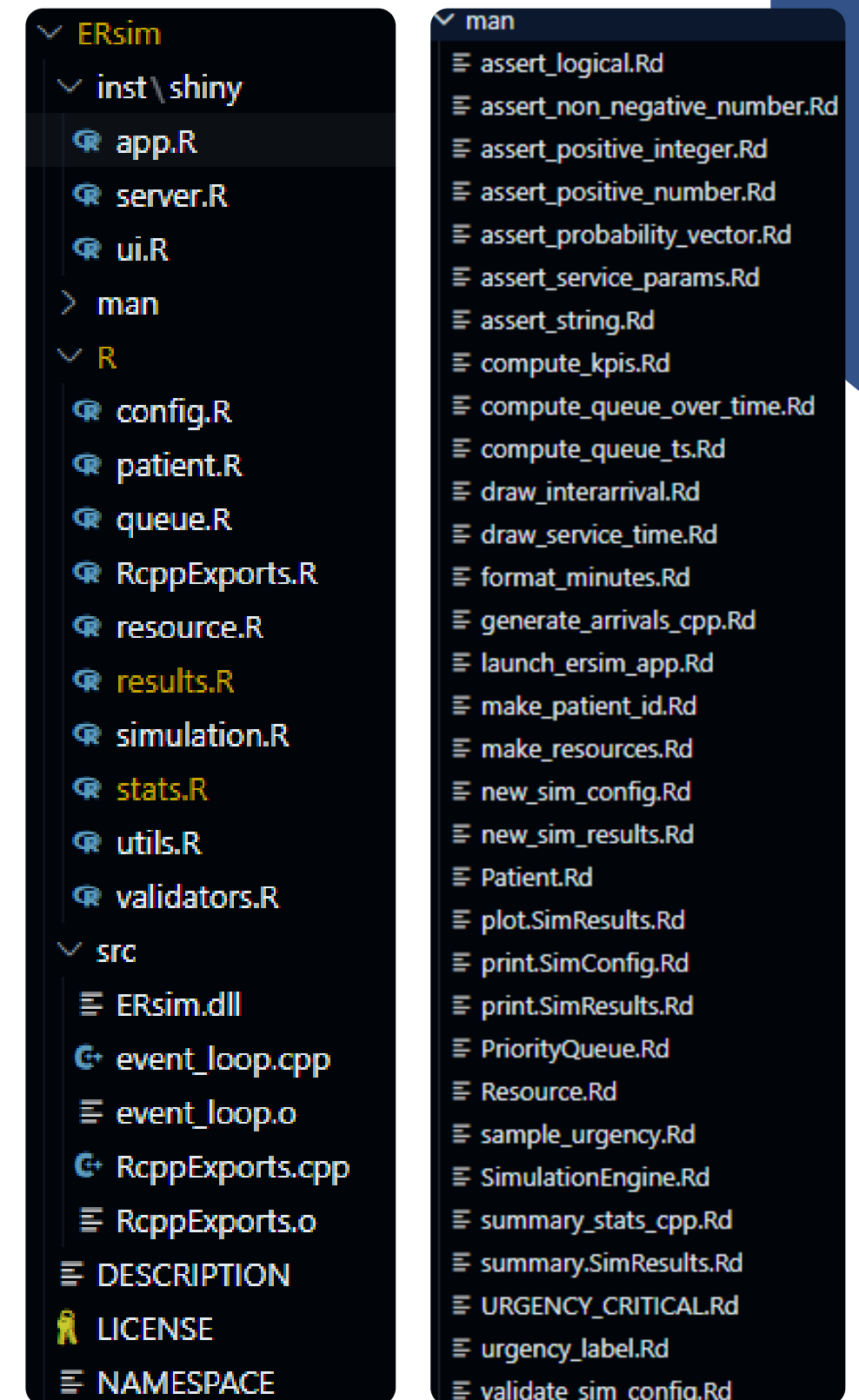
```
# Generated by roxygen2: do not edit by hand
# Run devtools::document() to regenerate

useDynLib(ERSim, .registration = TRUE)
importFrom(Rcpp, sourceCpp)

export(new_sim_config)
export(SimulationEngine)
export(Patient)
export(PriorityQueue)
export(Resource)
export(compute_kpis)
export(launch_ersim_app)

S3method(print, SimConfig)
S3method(print, SimResults)
S3method(summary, SimResults)
S3method(plot, SimResults)

importFrom(R6, R6Class)
importFrom(ggplot2, ggplot, aes, geom_histogram, geom_line, geom_bar,
            facet_wrap, labs, theme_minimal, theme, element_text,
            scale_fill_manual, coord_flip)
importFrom(dplyr, group_by, summarise, mutate, filter, arrange, n)
importFrom(scales, percent)
```





LIVE DEMO



THANK YOU!